

1. Core Concept Breakdown

A. React Native Styling Fundamentals

React Native does **not** use real CSS or HTML DOM elements. Instead, layout and styling are driven by JavaScript objects that mimic CSS properties.

- **CSS-Inspired, Not CSS:** While properties look like CSS (e.g., `backgroundColor`, `fontSize`), they are written in **camelCase** rather than **kebab-case**. Values are usually standard numbers representing Density-Independent Pixels (dp) rather than raw string values with `px`.
- **StyleSheet.create:** Instead of passing raw inline JavaScript objects on every render, React Native provides `StyleSheet.create`.
 - **Performance:** It registers styles internally, giving components numeric IDs instead of creating new JavaScript objects on every render pass.
 - **Validation:** It provides earlier, clearer error messages for misspelled properties or invalid values during execution.

B. The Flexbox Layout Engine

React Native relies **exclusively on Flexbox** for user interface layout. There is no CSS Grid, display table, or element floating.

1. The Key Difference: `flexDirection`

- **Web Default:** `flexDirection: 'row'` (items line up horizontally).
- **React Native Default:** `flexDirection: 'column'` (items stack vertically).
- **Why?** Mobile app screens are vertical scroll views by nature, making vertical stacking the logical default.

2. Main Axis vs. Cross Axis

Flexbox operates on two directions depending on your `flexDirection`:

Property	<code>flexDirection: 'column'</code> (Default) TSX	<code>flexDirection: 'row'</code> TSX
Main Axis (Controlled by <code>justifyContent</code>)	Vertical (top to bottom)	Horizontal (left to right)
Cross Axis (Controlled by <code>alignItems</code>)	Horizontal (left to right)	Vertical (top to bottom)

- **justifyContent:** Aligns items along the **Main Axis** (e.g., flex-start, center, flex-end, space-between, space-around, space-evenly).
- **alignItems:** Aligns items along the **Cross Axis** (e.g., flex-start, center, flex-end, stretch).
- **alignSelf:** Overrides the parent component's `alignItems` setting for a single, specific child element.
- **flex: number:** Defines how much available space an item should take up relative to its siblings. For instance, `flex: 1` on a root element instructs it to expand and fill the entire available screen height/width.

C. Responsive Design & Screen API Metrics

Mobile devices come in thousands of unique screen shapes, densities, and orientations. React Native offers tools to make layouts dynamic:

1. **`Dimensions.get('window')`:**
 - A synchronous API to obtain the initial width and height of the window frame.
 - **Gotcha:** Values fetched via `Dimensions.get('window')` are static upon component instantiation. If a user rotates their device, the layout won't automatically update unless an event listener is explicitly attached.
2. **`useWindowDimensions()` Hook:**
 - The modern React Hook for dynamic responsive layouts.
 - It automatically subscribes to layout changes (orientation switches, split-screen resizes) and triggers a re-render with updated `width` and `height` dimensions.
3. **PixelRatio API:**
 - Mobile device screens differ in pixel density (DP/DPI vs physical device pixels).
 - `PixelRatio.get()` returns the device pixel density ratio (e.g., 1, 2, 3).
 - `StyleSheet.hairlineWidth` uses `PixelRatio` under the hood to calculate a border width exactly equal to 1 physical pixel on any screen.
 - Crucial for serving crisp, correctly-sized bitmap image assets based on the device scale.

2. Code Examples & Detailed Explanations

Below are the practical component implementations explaining these concepts in detail.

Snippet 1: Basic App Setup (`App.js`)

Demonstrates basic view rendering and styling using `StyleSheet.create`.

TypeScript

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

const App = () => {
  return (
```

```

    <View style={styles.container}>
      <Text style={styles.title}>Hello World</Text>
      <Text style={styles.subtitle}>This is React Native</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1, // Fills the whole screen
    backgroundColor: '#807b7b',
    justifyContent: 'center', // Centers children along column axis
    (vertically)
    alignItems: 'center',      // Centers children along cross axis
    (horizontally)
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#f8d5d5',
  },
  subtitle: {
    fontSize: 16,
    marginTop: 10,
    color: 'ffffff',
  },
});

export default App;

```

- **Explanation:** View behaves like a <div>, and Text handles text display. Notice flex: 1 makes container occupy all space on the device display, while justifyContent: 'center' and alignItems: 'center' position the text elements directly in the middle of the screen.

Snippet 2: Main Axis & Layout Axes (AxisExample.js)

Demonstrates how shifting flexDirection changes which physical directions justifyContent and alignItems act upon.

TypeScript

```

import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

function AxisExample() {
  return (
    <View style={styles.container}>
      /* Column Layout (Default) */
      <View style={styles.columnLayout}>
        <View style={styles.item} />
        <View style={styles.item} />
      </View>

      /* Row Layout */
    </View>
  );
}

```

```

        <View style={styles.rowLayout}>
          <View style={styles.item} />
          <View style={styles.item} />
        </View>
      </View>
    );
  }

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
  columnLayout: {
    flexDirection: 'column', // Main axis = vertical, Cross axis = horizontal
    justifyContent: 'space-evenly', // Spacing applied vertically
    alignItems: 'center', // Elements centered horizontally
    height: 200,
    backgroundColor: '#532020',
    marginBottom: 20,
  },
  rowLayout: {
    flexDirection: 'row', // Main axis = horizontal, Cross axis = vertical
    justifyContent: 'space-around', // Spacing applied horizontally
    alignItems: 'center', // Elements centered vertically
    height: 200,
    backgroundColor: '#1f0808',
  },
  item: {
    width: 50,
    height: 50,
    backgroundColor: 'purple',
  },
});

export default AxisExample;

```

- **Key Takeaway:** Notice how changing `flexDirection` from `column` to `row` flips the roles of `justifyContent` and `alignItems`.

Snippet 3: Flex Ratios and `alignSelf` (`FlexProperties.js`)

Demonstrates proportional flex sizing (`flex: 1` vs `flex: 2`) and overriding alignment with `alignSelf`.

TypeScript

```

import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

const FlexProperties = () => {
  return (
    <View style={styles.container}>
      { /* flex: 1 takes 1 unit of available space */ }
      <View style={styles.flexOne}>

```

```

    <Text>flex: 1</Text>
  </View>

  {/* flex: 2 takes 2 units of space (twice as much as flex: 1) */}
  <View style={styles.flexTwo}>
    <Text>flex: 2</Text>
  </View>

  {/* alignSelf overrides the container's cross-axis alignment */}
  <View style={styles.alignSelfExample}>
    <View style={styles.alignSelfStart}>
      <Text>Start</Text>
    </View>
    <View style={styles.alignSelfCenter}>
      <Text>Center</Text>
    </View>
    <View style={styles.alignSelfEnd}>
      <Text>End</Text>
    </View>
  </View>
</View>
);
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 10,
  },
  flexOne: {
    flex: 1, // Occupies 1/3 of total vertical space
    backgroundColor: 'lightblue',
    justifyContent: 'center',
    alignItems: 'center',
  },
  flexTwo: {
    flex: 2, // Occupies 2/3 of total vertical space
    backgroundColor: 'lightgreen',
    justifyContent: 'center',
    alignItems: 'center',
  },
  alignSelfExample: {
    flexDirection: 'row',
    height: 100,
    backgroundColor: '#cf7070',
    alignItems: 'center', // Default cross-axis alignment for children
    marginTop: 10,
  },
  alignSelfStart: {
    alignSelf: 'flex-start', // Pushes element to top of container
    backgroundColor: 'orange',
    padding: 10,
  },
  alignSelfCenter: {
    alignSelf: 'center', // Positions element in center
    backgroundColor: 'pink',
    padding: 10,
  },
});

```

```

    },
    alignSelfEnd: {
      alignSelf: 'flex-end', // Pushes element to bottom of container
      backgroundColor: 'teal',
      padding: 10,
    },
  },
});

export default FlexProperties;

```

Snippet 4: Legacy Static Dimensions (`DimensionsExample.js`)

Calculates proportions manually using static window measurements.

TypeScript

```

import React from "react";
import { Dimensions, StyleSheet, Text, View } from "react-native";

// Static measurements captured once at file execution time
const window = Dimensions.get("window");
const { width, height } = window;

const DimensionsExample = () => {
  return (
    <View style={styles.container}>
      <Text>Window Width: {width}</Text>
      <Text>Window Height: {height}</Text>

      {/* Box sizing calculated as percentages of total screen width and
height */}
      <View style={styles.responsiveBox1}>
        <Text>Responsive Box1</Text>
      </View>
      <View style={styles.responsiveBox2}>
        <Text>Responsive Box2</Text>
      </View>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
  },
  responsiveBox1: {
    width: width * 0.8, // Exactly 80% of screen width
    height: height * 0.2, // Exactly 20% of screen height
    backgroundColor: "coral",
    justifyContent: "center",
    alignItems: "center",
    marginTop: 20,
    borderRadius: 10,
  },
});

```

```

    responsiveBox2: {
      width: width * 0.8,
      height: height * 0.2,
      backgroundColor: "#f76c6c",
      justifyContent: "center",
      alignItems: "center",
      marginTop: 10,
      borderRadius: 10,
    },
  });

export default DimensionsExample;

```

Snippet 5: Dynamic Device Orientation Listeners (`DimensionsWithRotation.js`)

Demonstrates how to listen for screen size changes (like device rotation) using event listeners.

TypeScript

```

import React, { useState, useEffect } from 'react';
import { View, Text, Dimensions, StyleSheet } from 'react-native';

const DimensionsWithRotation = () => {
  // Store window dimensions in component state
  const [window, setWindow] = useState(Dimensions.get('window'));

  useEffect(() => {
    // Add event listener to capture dimensions when device rotates
    const subscription = Dimensions.addEventListener('change', ({ window })
=> {
      setWindow(window);
    });

    // Cleanup event listener when component unmounts
    return () => subscription.remove();
  }, []);

  return (
    <View style={styles.container}>
      <Text>Width: {window.width}</Text>
      <Text>Height: {window.height}</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
});

export default DimensionsWithRotation;

```

Snippet 6: Modern Screen Metrics with `useWindowDimensions` (`UseWindowDimensionsExample.js`)

Shows how to handle responsive styling and screen orientations automatically using hooks.

TypeScript

```
import React from 'react';
import { View, Text, useWindowDimensions, StyleSheet } from 'react-native';

const UseWindowDimensionsExample = () => {
  // Automatically updates values and triggers re-renders on resize/rotation
  const { width, height, scale, fontScale } = useWindowDimensions();

  const isLandscape = width > height;
  const isTablet = width > 768;

  return (
    <View style={styles.container}>
      <Text>Width: {width}</Text>
      <Text>Height: {height}</Text>
      <Text>Scale: {scale}</Text>
      <Text>Font Scale: {fontScale}</Text>
      <Text>Orientation: {isLandscape ? 'Landscape' : 'Portrait'}</Text>
      <Text>Device: {isTablet ? 'Tablet' : 'Phone'}</Text>

      {/* Apply dynamic box styles based on orientation */}
      <View style={[
        styles.responsiveBox,
        isLandscape ? styles.landscapeBox : styles.portraitBox,
      ]}>
        <Text>{isLandscape ? '🖥️' : '📱'}</Text>
        <Text>{isLandscape ? 'Landscape Mode' : 'Portrait Mode'}</Text>
      </View>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  responsiveBox: {
    marginTop: 20,
    padding: 20,
    borderRadius: 10,
    justifyContent: 'center',
    alignItems: 'center',
  },
  portraitBox: {
    width: 200,
    height: 300,
    backgroundColor: '#4CAF50',
  },
});
```



```

    },
    landscapeBox: {
      width: 300,
      height: 150,
      backgroundColor: '#FF9800',
    },
  });

export default UseWindowDimensionsExample;

```

Snippet 7: Physical Pixel Density & Scaling (PixelRatioExample.js)

Using `PixelRatio` to draw crisp, single-physical-pixel borders and calculate image scaling factors.

TypeScript

```

import React from 'react';
import { View, Text, PixelRatio, StyleSheet } from 'react-native';

const PixelRatioExample = () => {
  const pixelRatio = PixelRatio.get();
  const fontScale = PixelRatio.getFontScale();
  const pixelDensity = PixelRatio.getPixelSizeForLayoutSize(1);

  // Calculates image render width scaled up by device density
  const imageSize = 50 * pixelRatio;

  return (
    <View style={styles.container}>
      <Text>Pixel Ratio: {pixelRatio}</Text>
      <Text>Font Scale: {fontScale}</Text>
      <Text>ldp = {pixelDensity} physical pixels</Text>
      <Text>Hairline width: {StyleSheet.hairlineWidth}px</Text>
      <Text>Image target size: {imageSize}px</Text>

      {/* Hairline box draws a 1-physical-pixel thick line on any screen */}
      <View style={styles.hairlineBox}>
        <Text>Hairline border</Text>
      </View>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  hairlineBox: {
    borderWidth: StyleSheet.hairlineWidth, // Precise 1 physical pixel width
    borderColor: 'red',
    padding: 20,
    marginTop: 20,
  },
});

```

```

        backgroundColor: '#e7d8d8',
    },
});

export default PixelRatioExample;

```

Snippet 8: Complete Responsive Component (ProfileCard.js)

Combines dynamic screen break points, custom scaling, and adaptive layouts into a single production-ready component.

TypeScript

```

import React from 'react';
import {
    View,
    Text,
    Image,
    StyleSheet,
    useWindowDimensions,
} from 'react-native';

interface ProfileCardProps {
    name: string;
    role: string;
    avatarUrl: string;
}

const ProfileCard: React.FC<ProfileCardProps> = ({ name, role, avatarUrl })
=> {
    const { width } = useWindowDimensions();

    // Define breakpoints
    const isTablet = width > 768;
    const isSmallPhone = width < 375;

    // Responsive styling decisions based on device screen size
    const cardPadding = isTablet ? 30 : isSmallPhone ? 10 : 20;
    const avatarSize = isTablet ? 120 : isSmallPhone ? 60 : 80;
    const titleSize = isTablet ? 28 : isSmallPhone ? 16 : 22;
    const subtitleSize = isTablet ? 18 : isSmallPhone ? 12 : 14;

    return (
        <View
            style={[
                styles.card,
                {
                    padding: cardPadding,
                    flexDirection: isTablet ? 'row' : 'column', // Horizontal layout
                                                                    for tablets, vertical for phones
                    alignItems: 'center',
                },
            ]}
        >
            <View
                style={[

```

```

        styles.avatarContainer,
        {
            width: avatarSize,
            height: avatarSize,
            marginBottom: isTablet ? 0 : 15,
            marginRight: isTablet ? 20 : 0,
        },
    ]}
    >
    <Image
        source={{ uri: avatarUrl }}
        style={{
            width: avatarSize,
            height: avatarSize,
            borderRadius: avatarSize / 2, // Perfect circle shape
        }}
    />
</View>

    <View style={[styles.info, { alignItems: isTablet ? 'flex-start' :
'center' }]}>
        <Text style={[styles.name, { fontSize: titleSize }]}>{name}</Text>
        <Text style={[styles.role, { fontSize: subtitleSize }]}>{role}</Text>
    </View>
</View>
    );
};

const styles = StyleSheet.create({
  card: {
    backgroundColor: 'white',
    borderRadius: 12,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 8,
    elevation: 5, // Elevation handles drop shadows on Android
    margin: 16,
  },
  avatarContainer: {
    borderRadius: 100,
    overflow: 'hidden',
    backgroundColor: '#e0e0e0',
  },
  info: {
    flex: 1,
  },
  name: {
    fontWeight: 'bold',
    color: '#333',
    marginBottom: 4,
  },
  role: {
    color: '#666',
  },
});

```

```
export default ProfileCard;
```